Outcomes & Reputation

How the network learns what's true — recommendation validation & the reputation math · engine v1 · July 2026

1 · The question this answers

Maria recommends an immigration lawyer. Months later, the network "knows" this lawyer is good — good enough to rank them higher and to reward Maria. **How does the network find that out?** There is no voting, no likes, no admin decision, and no AI verdict. The mechanism is a single primitive: the **outcome**.

> **An outcome is a real-world result.** Someone acted on a recommendation — hired the lawyer, ate at the restaurant, visited the dentist — and then reported how it actually went, on a scale from −1 (bad) to +1 (great). Outcomes are the only currency of truth in Trusty.

2 · The validation flow, step by step

```
Maria publishes a recommendation for lawyer L → Maria: +2 contribution (creation)
↓
Alex discovers it (trust search, or he Listens to Maria)
↓
Alex hires the lawyer — real life happens
↓
Alex reports the outcome (value +1) → Alex: +1 contribution (validation)
↓
The single report moves three independent dials:
① an immutable outcome node attaches to the rec → the rec's consensus grows
② the append-only outcome log gains a row → lawyer L's REPUTATION updates
③ positive + from someone ≠ Maria → Maria: +3 (recommendation_validated)
```

Outcomes are **immutable and additive** — they never rewrite the recommendation, they accumulate beside it. Contradictory evidence is preserved, not discarded: a later −1 outcome sits in history next to the earlier +1, and consensus is computed on read.

3 · Does Maria get more score? Yes — the most the engine ever gives

`recommendation_validated` is worth **3** — more than creating the recommendation itself (2). Deliberately: an independent human verifying in the real world that your knowledge was good is the hardest signal to fake, so it pays the most.

The three guardrails

GUARDRAIL	RULE	WHY
Independence	One `+3` per *distinct person* per recommendation. Alex reporting five glowing outcomes on the same rec = +3 once, not +15. Three *different* people = +9.	People count, volume doesn't — no farming a friend's repeat clicks.
Positive only (for the author)	A negative outcome earns Maria nothing. The *reporter* still earns +1 either way.	Honest bad news also reduces uncertainty — the reporter helped the network; the author's bad advice didn't.
No self-validation	Maria's own outcomes on her own recommendation are excluded.	Obvious.

Timing note: the +3 lands with the outcome's own date, then flows through the normal scoring machinery — full value in the never-decaying lifetime component, recency-weighted in momentum, damped if several land the same day.

4 · Reputation — whose is it, and what is it?

Reputation belongs to the **provider** (lawyer L), not to Maria. It answers: "*how reliably has this provider produced good outcomes, in this topic?*" Three frozen properties:

- **Derived, never stored.** There is no reputation column anywhere. It is recomputed from the outcome log on every read — and can therefore always be replayed under a better model.
- **Never user-editable.** Not by the provider, not by an admin. Only evidence moves it.
- **Topic-scoped.** A brilliant immigration-lawyer reputation says exactly nothing about the same person as a tax advisor. (Child topics roll up through the topic hierarchy.)

5 · The reputation math — four steps

① Time decay

Each outcome's weight halves every **180 days** (`halfLifeDays` , default 180). Fresh evidence counts fully; a three-year-old outcome still counts, just faintly. Reputation reflects the provider as they are *now*, without amnesia.

```
weight = 0.5 ^ (age_in_days / 180)
```

② Weighted mean of outcome values

Each outcome value (−1…+1) is multiplied by its decay weight and averaged.

③ Bayesian shrinkage — one review can't make a star

```
score = (Σ weight·value + K·prior) / (Σ weight + K) with K = 3, prior = 0
```

The **K = 3** acts like three invisible, perfectly neutral outcomes sitting in the average. A single +1.0 outcome therefore produces a score of 1/(1+3) = **0.25**, not 1.0. Scores move slowly and stably — the spec's "favor stability over volatility". One outcome can never dominate.

④ Confidence from independent people — never a vote count

```
confidence = 1 − e^(−distinct_contributors / 2)
```

Confidence grows with the number of *different people* who reported, and saturates. Ten outcomes from one enthusiastic person ≈ one contributor ≈ 39% confidence. The same ten spread over ten people ≈ 99%.

DISTINCT CONTRIBUTORS	1	2	3	5	10
CONFIDENCE	39%	63%	78%	92%	99%

The ranking-ready number

```
effective = score × confidence
```

Low evidence ⇒ near-neutral, no matter how glowing the few reports were. This is the number the ranking pipeline consumes — as *one* config-weighted factor among several (recommendation quality, outcome consensus, relationship strength, personalization), never as the whole answer.

6 · Worked example — Maria's lawyer

Two clients so far: Alex reports **+1.0** today; Sara reported **+0.8** sixty days ago.

STEP	COMPUTATION	RESULT
Sara's decay weight	0.5^(60/180)	0.79
Weighted evidence	1.0×1.0 + 0.79×0.8	1.63
Score (with shrinkage K=3)	1.63 / (1.79 + 3)	**0.34**
Confidence (2 people)	1 − e^(−2/2)	**0.63**
Effective reputation ("lawyer")	0.34 × 0.63	**≈ 0.22**

Two happy clients get the lawyer to a modest 0.22 — not to the top. Building a strong reputation takes **many independent, sustained** positive outcomes; and it decays gracefully if the quality stops.

Meanwhile, from the same two reports: Maria earned +3 twice (two independent validators → +6 contribution), Alex and Sara each earned +1, and the recommendation itself shows a 2-person positive consensus. Four different ledgers, one piece of evidence each — none of them purchasable.

7 · What this deliberately is NOT

> **Structural absences.** Reputation has no input for: star ratings without an acted-on outcome, likes, views, follower counts, payment or subscription, provider self-reports, admin overrides, or AI judgments. The signature of the function is `(outcome samples, time) → score` — nothing else can reach it.

- **Reputation ≠ Trust.** Trust is *your* directed, topic-scoped relationship graph — who *you* rely on, transferred along your own paths. Reputation is the network-wide evidence view of a provider. They are computed by different modules from different edges and never write to each other.
- **Reputation ≠ Contribution.** The lawyer's reputation rising does nothing to Maria's score except via the explicit `recommendation_validated` event — and Maria's contribution never feeds back into the lawyer's reputation.
- **Listening moves neither.** Gaining a thousand listeners changes no reputation and no contribution.

8 · Configuration

KNOB	DEFAULT	CONTROLS
`halfLifeDays`	180	Outcome decay — how fast old evidence fades
`priorStrength (K)`	3	Shrinkage — how many "invisible neutral outcomes" temper the mean
`prior`	0	The neutral point scores shrink toward
`consensusK`	2	How quickly confidence saturates with independent contributors

All defaults live in `packages/graph-core/src/reputation.ts` (`REPUTATION_DEFAULTS`); the SQL evidence source is the append-only `trust_outcomes` log.

Trusty · Outcomes & Reputation · generated July 2026 · companion to the Contribution/Credits/Capacity glossary · sources: `packages/graph-core/src/reputation.ts` , `recommendations.ts` , `services/graph-service/src/contribution.ts` , `trustLog.ts` , GitHub issues #9–#11, #20–#23.